

CAMPUS PRINT HUB

TECHNICAL & OPERATIONAL CHANGE REGISTER (RC1.5)

Generated on: 8/7/2026 | Author: Technical Lead Architect

1. EXECUTIVE SUMMARY

This engineering document contains the final change register implemented during the RC1.5 release cycle of Campus Print Hub. The changes address critical hardware synchronization issues, connection staleness under proxy conditions, route authorization security bypasses, and operational launch orchestration. These revisions stabilize the codebase for live deployment without modifying the validated core printing, routing, or database pipelines.

2. AGENT CONNECTION & NETWORK ARCHITECTURE (THE DUAL-TUNNEL)

Why:

Previously, the Print Agent (running on the Shop PC) connected to the primary Cloudflare Tunnel on port 3000, which serves the Vite Development Server. Vite proxied requests under '/api' to the Express server (port 3001). However, Vite's built-in dev proxy is designed for short-lived HTTP requests. The print agent relies on two long-running TCP connections: heartbeats (every 30 seconds) and Server-Sent Events (SSE) (continuous listener via '/api/jobs/stream'). The Vite proxy repeatedly closed these sockets due to activity timeouts, marking the agent offline.

How:

Introduced a dual-tunnel network architecture. We launched a direct API tunnel to port 3001. The agent's 'config.json' was updated to point directly to this backend tunnel, completely bypassing the Vite proxy.

Where:

Client Configuration: print-client/config.json

Importance:

Eliminates heartbeat dropouts and connection timeouts, ensuring the agent remains permanently connected to the print queue in real time.

3. API AUTHORIZATION & PUBLIC ROUTE ACCESS

Why:

The Express backend protects the '/api/printer/settings' route using the 'requireAdmin' middleware. The React frontend fetches this route on mount to update the printer status indicator. Since the Student Portal is a public-facing interface, students do not possess an admin token. Consequently, their browser threw 401 Unauthorized errors silently, and the print indicator fell back to 'OFFLINE' permanently, blocking uploads.

How:

Removed the 'requireAdmin' middleware from the GET '/api/printer/settings' endpoint. Since this endpoint only reads public printer indicators (status, speed, return time), it is safe to be read publicly. Write operations (like triggering scans) remain securely protected.

Where:

Backend: server/index.ts (Line 932)

Importance:

Restores synchronization. Public customers can now view when the shop is active and submit print jobs without authentication blocks.

4. FRONTEND STATE & SYNCHRONIZATION FIXES

Why:

Two issues prevented real-time status updates: 1) App.tsx was missing the Authorization header on its root fetch, preventing logged-in admins from viewing status on reload. 2) There was a property mismatch: React read 'agentLastHeartbeat' while the server returned 'lastHeartbeat'. 3) The Admin Dashboard B&W and Color cards were local variables that only updated after a manual scan, defaulting to offline on mount.

How:

1. Attached `Authorization: Bearer <token>` to the settings fetch in `App.tsx` and aligned the property key to `lastHeartbeat`.
2. Modified `AdminPortal.tsx` to call `/api/printer/settings` on mount and integrated it into the 3-second stats interval.

Where:

Frontend: src/App.tsx (L95-120) and src/components/AdminPortal.tsx (L349-385, L445)

Importance:

Ensures the admin interface shows the true ready state of printers without requiring manual scan polling.

5. OPERATIONAL ORCHESTRATOR (LAUNCHER DESIGN)

Why:

Shop operators are not developers. Requiring them to open VS Code, start multiple terminal windows, or execute Node commands manually represents a high risk of user error and system instability.

How:

Built a self-contained supervisor launcher daemon (running on port 3002). The operator double-clicks a single `.bat` file which launches the daemon and automatically opens their web browser to the controller panel. Clicking 'Start Hub' executes backend, frontend, and agent sequentially, verifies startup health and database registration, and redirects to the portal. Clicking 'Stop Hub' terminates child processes cleanly using Windows process tree taskkill commands.

Where:

Entry Batch: Start_Campus_Print.bat

Supervisor Code: launcher/server.js

HTML/CSS Dashboard: launcher/public/ (index.html, style.css, app.js)

Importance:

Simplifies daily operation to a single click, completely hiding the command line, scripts, and logs from the shop operator.

6. NETWORK PORT & API REGISTRY

Port 3000: Vite Frontend Server (Browser Access)

Port 3001: Express Backend API Server (Database / Tunnels)

Port 3002: Campus Print Launcher Daemon (Local UI)

GET /api/status - Returns running daemon states and tail logs (Launcher API)

POST /api/start - Spawns backend, frontend, and agent processes (Launcher API)

POST /api/stop - Terminates managed child processes (Launcher API)

GET /api/printer/settings - Fetches live shop readiness (Public API)